MASPOE's

MVP Poll application

3th BSc Business Engineering

Database systems & Data Structures

OVERVIEW

01
About MASPOE

02
Our Team

03
Beginning

04
Strategy

05
Result

06
Future plans

ABOUT MASPOE

- Belgian event-organizing organization
- Focused on sports, culture and entertainment
- Managing Sports & Events
- Bring people together through movement and shared experiences
- Vicaris Village, a 3-day festival they organize in Dendermonde

MASPOE
SPORTS & EVENTS

Our Team

Judith Ceuppens

General Manager

Louis Van Linden

Backend Engineer

Arjen Sap

Database Architect

Korneel Pennewaert

Frontend Developer

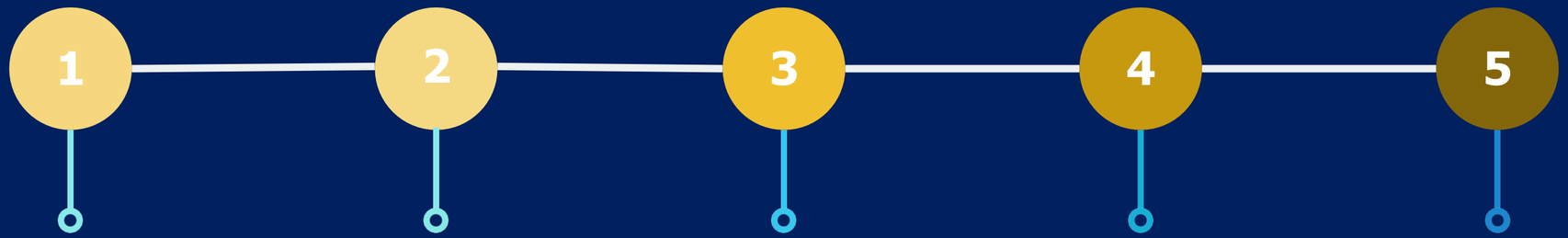
Kasper De Vos

Database Architect

Beginning

- Had to find own business
- Maspoe wanted site
- Settled on poll system ➔ didn’t know which artists to book

Strategy

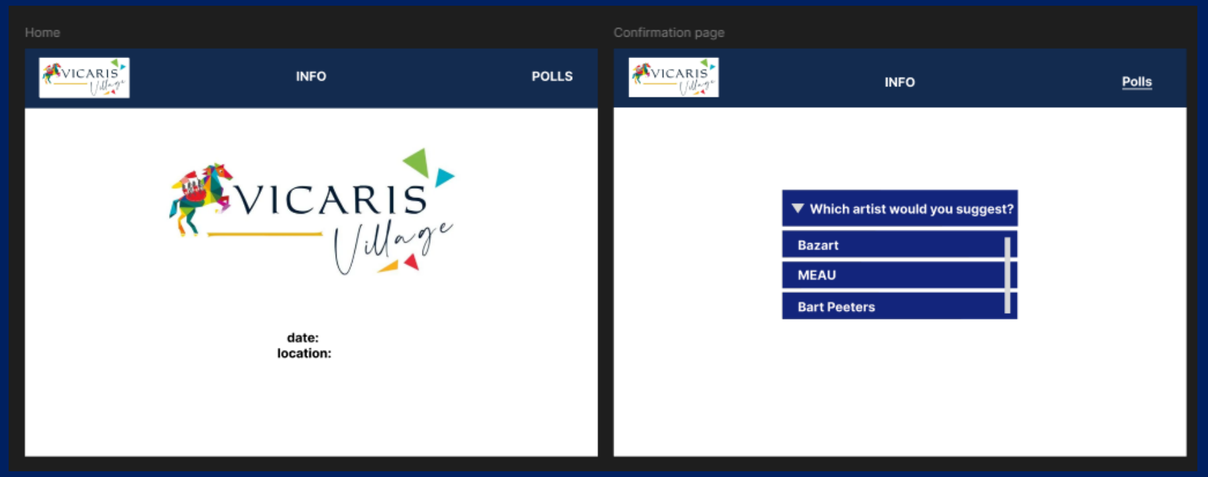


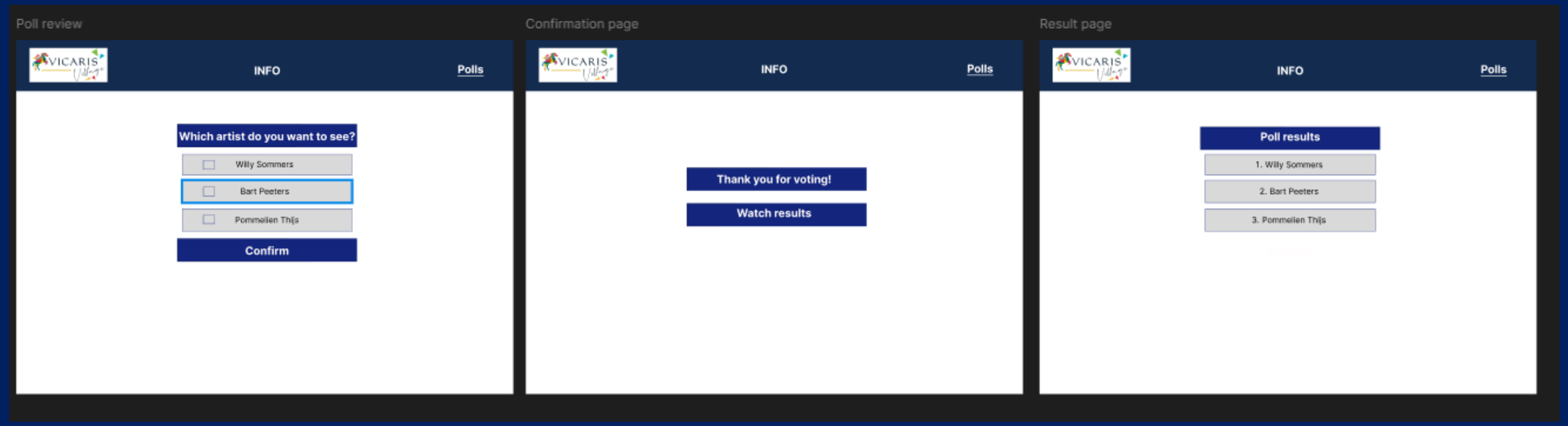
1. User Stories
2. ERD &DDL
3. Flask app
4. Complex algorithm
5. Presentation and finishing touch

User stories

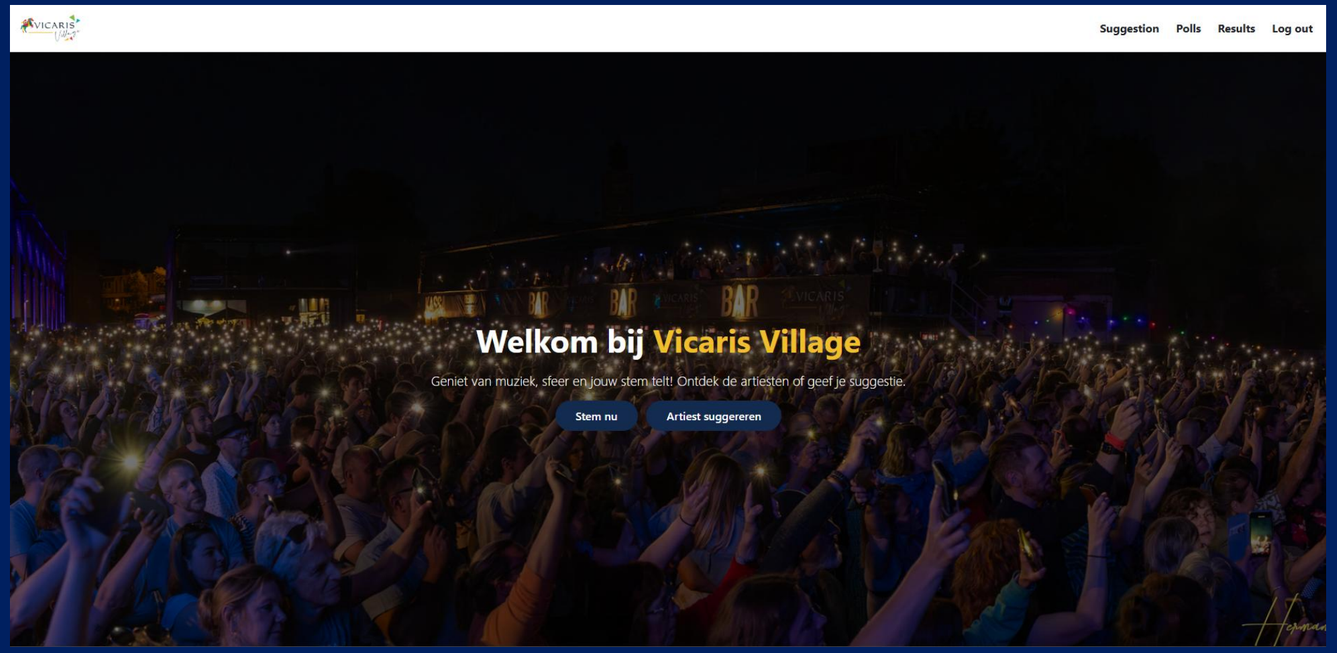
- As a user, I want to view available polls and submit a vote or suggestion, so that I can actively participate in the Vicaris Village selection process.
- Users can submit a vote via the polling interface
- Users can suggest an artist using a form
- As a user, I want to view clear and structured poll results and submitted suggestions, so that I can understand the collective preferences of the community.
- As an admin, I want to manage artists, editions and polls, so that the Vicaris Village application remains accurate, up to date and scalable for future events.

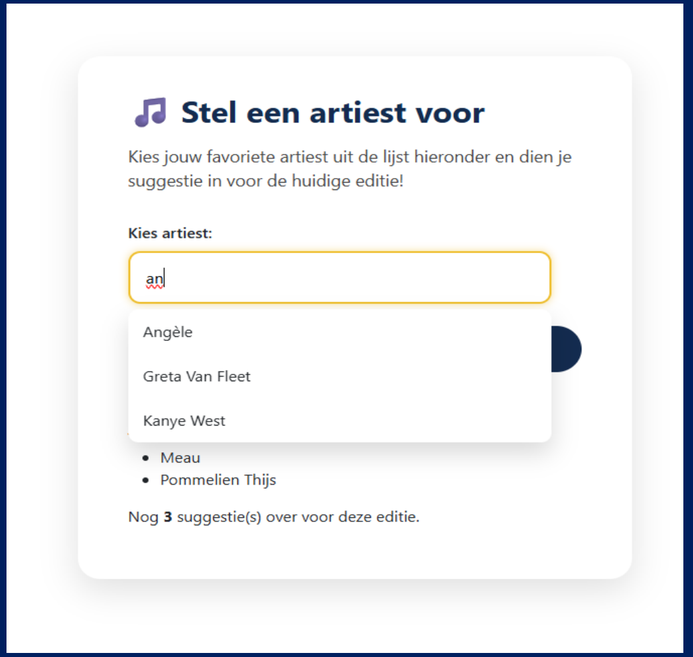
User Interface with Figma (initial)





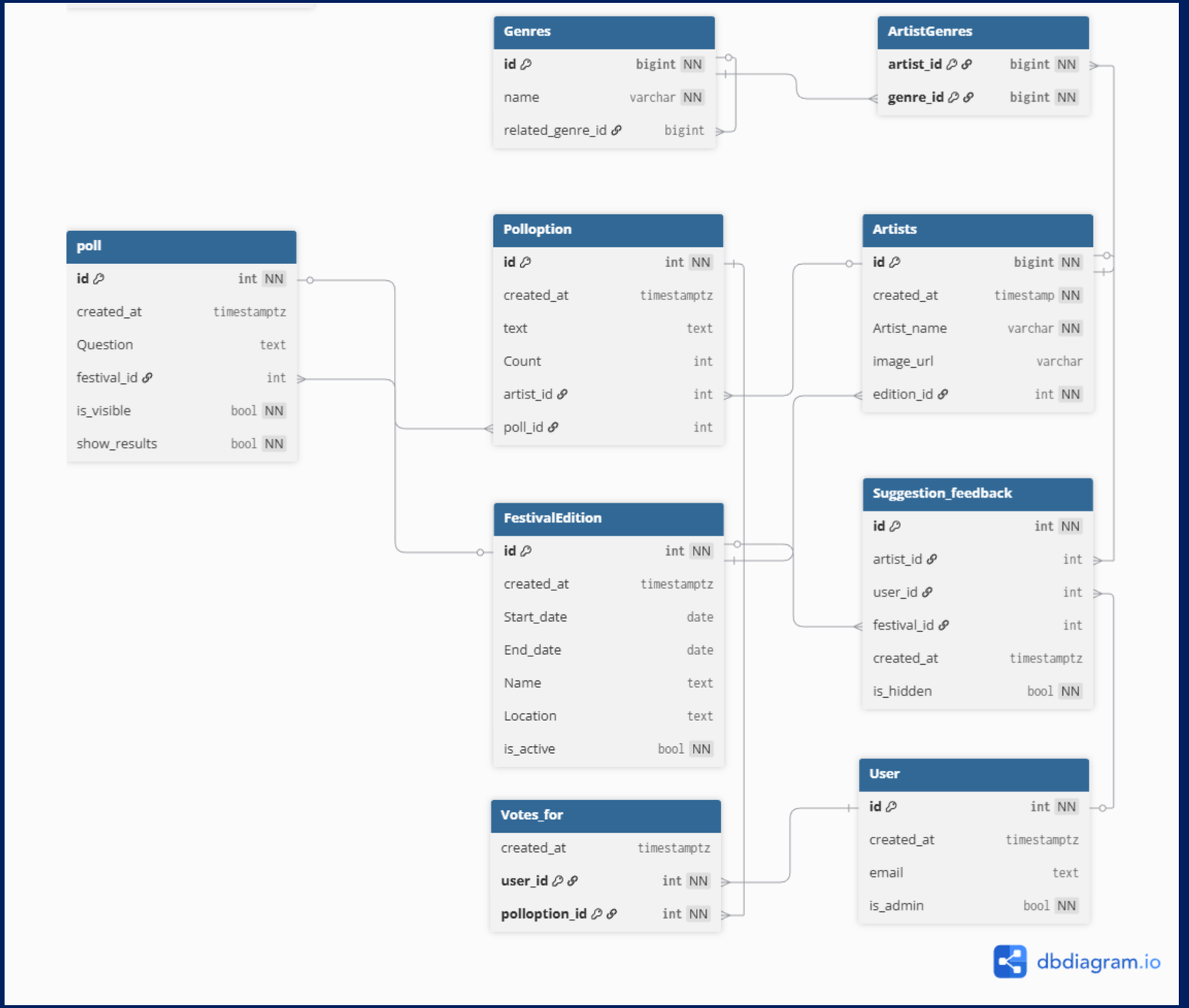
User Interface (now)





ERD

- 9 entities
- 42 attributes
- Each entity has 1 or more primary keys
- Relationships implemented using foreign keys
- N:M relationships resolved using junction tables



Festival Edition

- Our poll application works for multiple festivals
- Possible to adapt the edition

Overall architecture

- Flask *application factory* (`create_app()`)
- Central `models.py` using SQLAlchemy
- Blueprints in `app/routes/`
- HTML templates in `app/templates/`
- Static assets in `app/static/`
- Business logic in `app/services/`
- Helper utilities in `app/utils/`

Flask app

- `create_app()` loads configuration (`Config`)
- Initializes database (`db.init_app`)
- Enables Flask-Migrate
- Registers all blueprints
- Context processors:
  - `current_user`
  - `poll_visibility`
  - `suggestions_visibility`
  - `results_visibility`
- @app.before_request-hook: always an active poll

Routes

- `core.py` --> homepage
- `auth.py` --> register / login / logout
- `suggestions.py` -->artist suggestions (max. 5 per edition)
- `poll.py` --> poll loading, voting, and results
- `admin.py` --> admin dashboard
- `admin_editions.py` --> edition management

Database models

- `User` --> authentication + `is_admin`
- `FestivalEdition` --> edition details + active flag
- `Artists` — artists linked to an edition
- `Genres` + `ArtistGenres` --> many-to-many relationship
- `SuggestionFeedback` --> user suggestions per edition
- `Poll`, `Polloption`, `VotesFor` --> complete poll system

Services and business logic

- `poll.py` service:
  - `get_active_festival()`
  - `get_or_create_poll_for_edition()`
  - `get_or_create_active_poll()`
- `genre_profile.py` service:
  - Builds user music profile from suggestions
  - Generates personalized poll options
- `session.py` utility:
  - `get_session_user()`

Algorithmic support

- Builds a user genre profile based on suggestions
- Scores artists using genre similarity
- Selects top-ranked artists for personalised polls

Strenghts

- Useful for festivals
- Easy to use for customers
- Easy to filter artists for admin
- Upscaleable
- Everyone can vote
- Sorted into genres

Upscalability

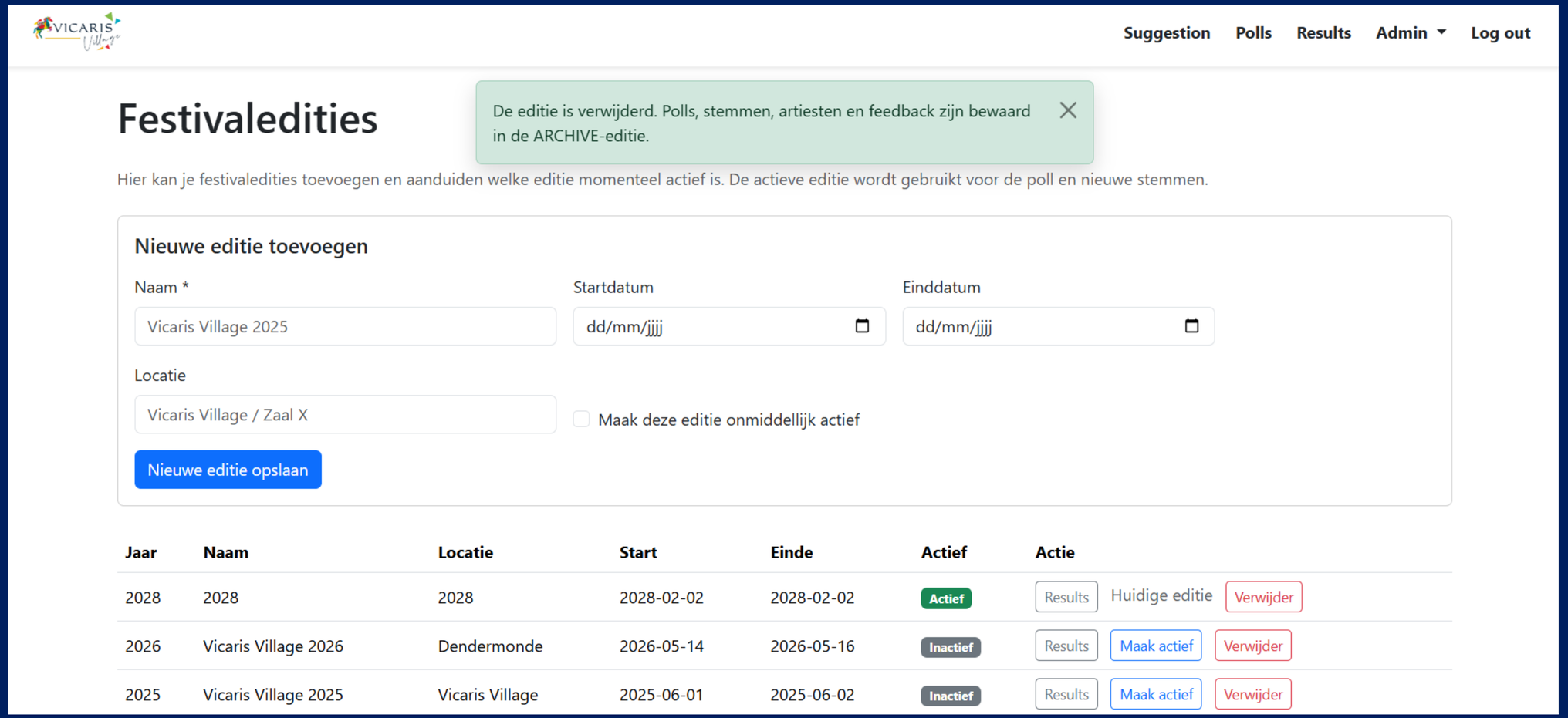
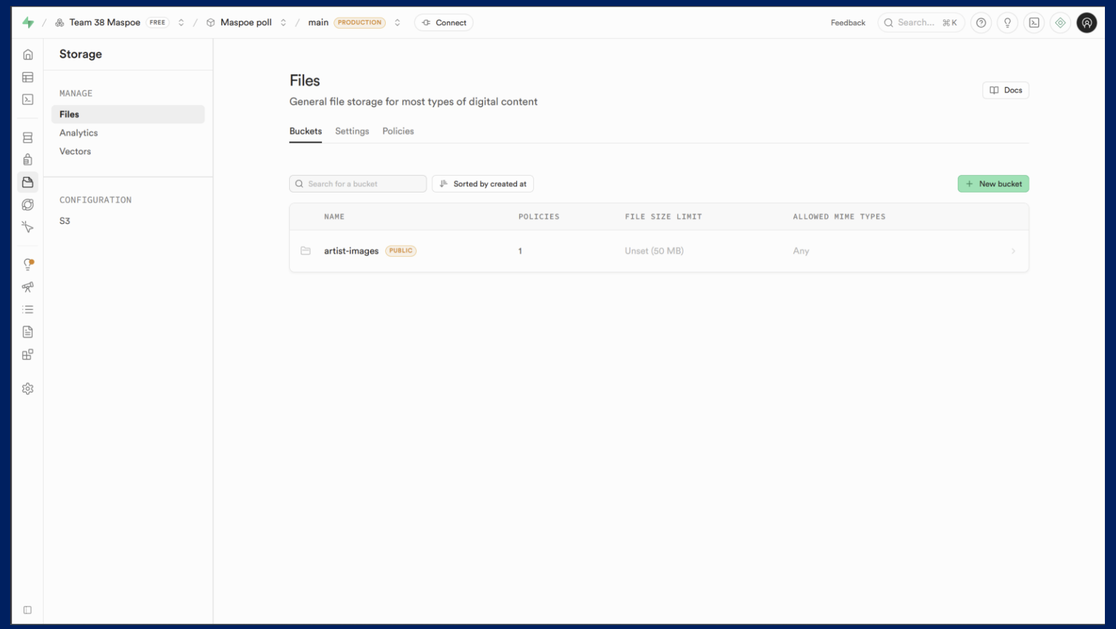
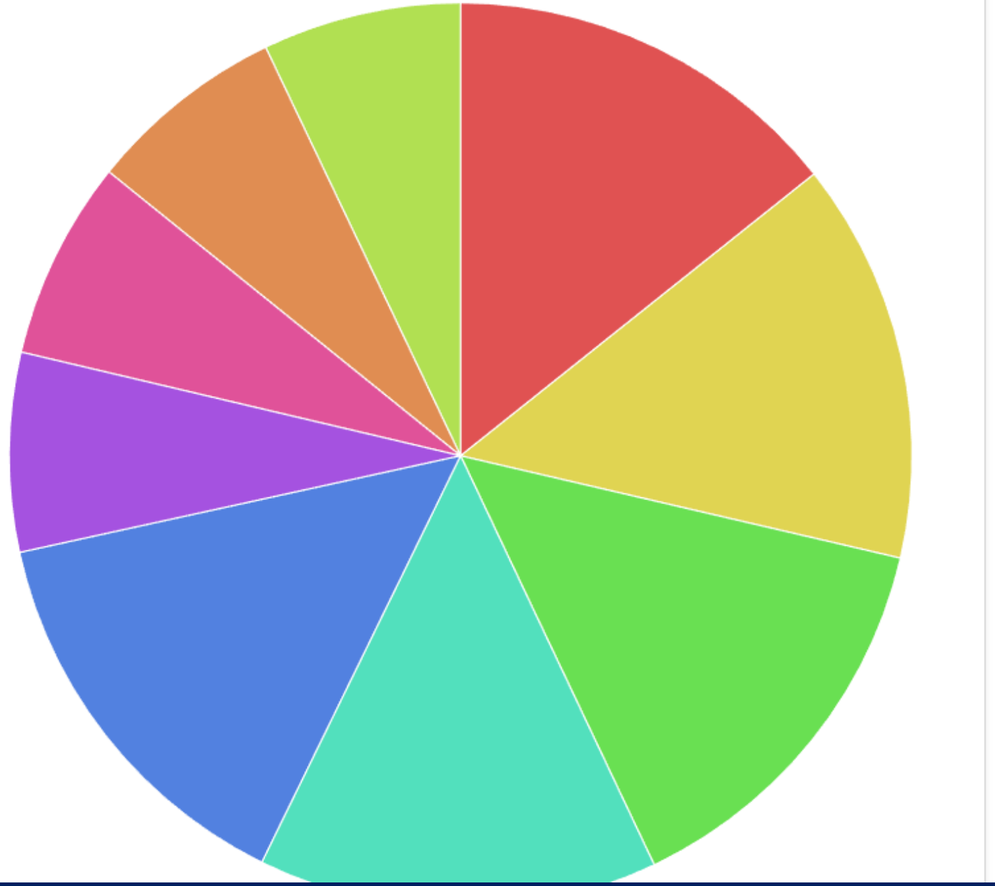


Image storage as an upscaling strategy



- **Current approach**
- Images stored on the filesystem
- Database only stores the relative image path
- Flask serves images via dynamic static routing
- **Upscaling option**
- Move images to a Supabase Storage Bucket
- Keep only the image URL/path in the database
- Improved scalability, performance and deployment flexibility

Result



Tekstueel overzicht (toegankelijkheidsfallback)

Hip-Hop	**14.3% (2)**
Pop	**14.3% (2)**
Rock	**14.3% (2)**
Singer-Songwriter	**14.3% (2)**
Vlaamse slagers	**14.3% (2)**
Alternative	**7.1% (1)**
Electronic	**7.1% (1)**
R&B	**7.1% (1)**
Rap	**7.1% (1)**

The future

- Making it more accessible for more festivals
- Making it easier for the organisers to edit
- Upscaling

Kanban

- Team 38 Maspoe - Miro
- Not possible anymore to edit

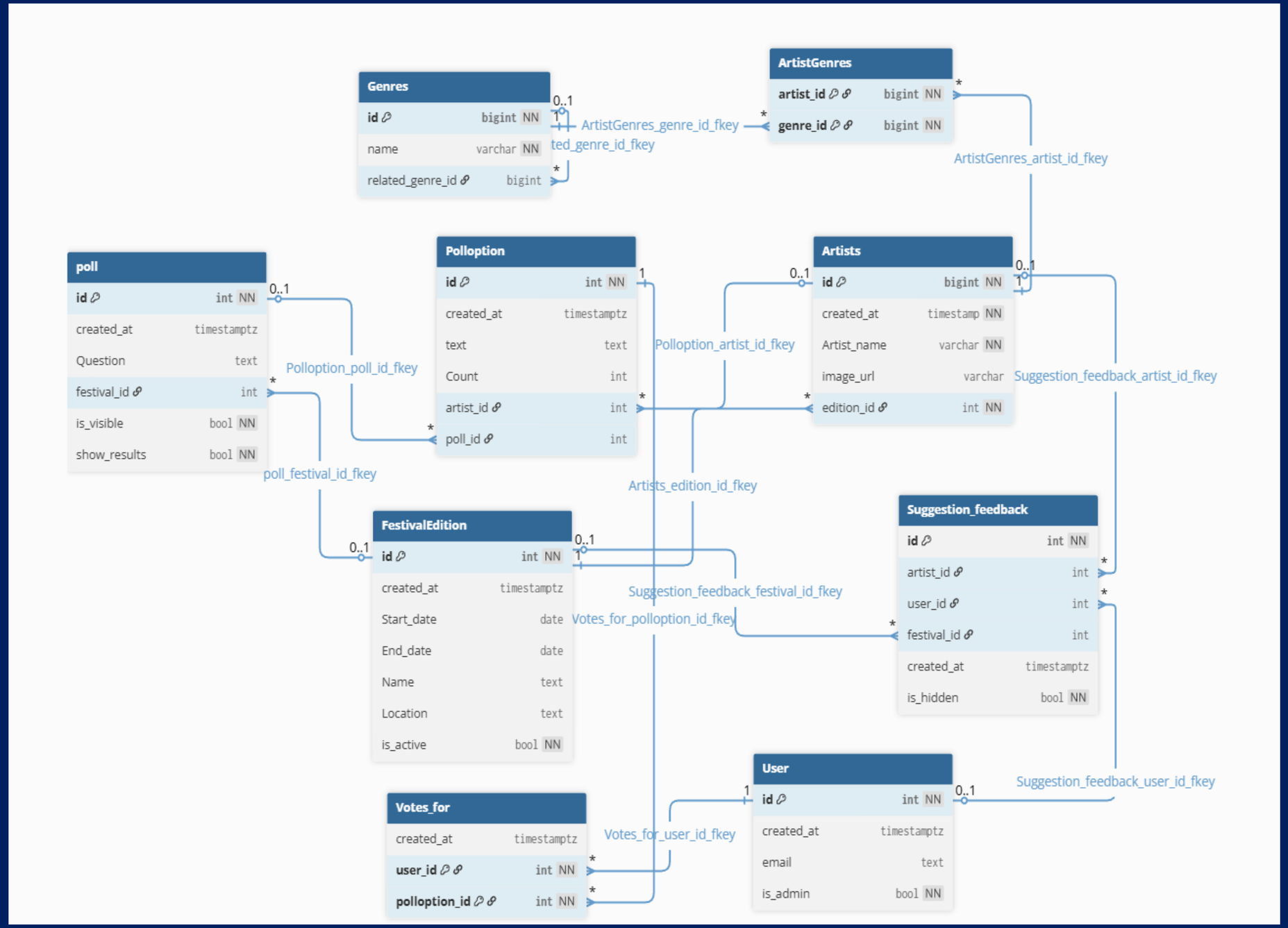
Thank you for listening

Group 38

Appendix

- Site Maspoe
- MVP render – polls
- Team 38 kanban - Miro

Q&A


Genres
id bigint NN
name varchar NN
related_genre_id bigint
ArtistGenres
artist_id bigint NN
genre_id bigint NN
ArtistGenres_genre_id_fkey
ted_genre_id_fkey
ArtistGenres_artist_id_fkey
poll
id int NN
created_at timestamptz
Question text
festival_id int
is_visible bool NN
show_results bool NN
Polloption
id int NN
created_at timestamptz
text text
Count int
artist_id int
poll_id int
Artists
id bigint NN
created_at timestamp NN
Artist_name varchar NN
image_url varchar
edition_id int NN
Polloption_poll_id_fkey
poll_festival_id_fkey
Polloption_artist_id_fkey
Artists_edition_id_fkey
Suggestion_feedback_artist_id_fkey
FestivalEdition
id int NN
created_at timestamptz
Start_date date
End_date date
Name text
Location text
is_active bool NN
Suggestion_feedback
id int NN
artist_id int
user_id int
festival_id int
created_at timestamptz
is_hidden bool NN
Suggestion_feedback_festival_id_fkey
Votes_for_polloption_id_fkey
User
id int NN
created_at timestamptz
email text
is_admin bool NN
Suggestion_feedback_user_id_fkey
Votes_for
created_at timestamptz
user_id int NN
polloption_id int NN
Votes_for_user_id_fkey

Algorithmic Support

app > routes > poll.py > ...

```python

@bp.get("/results")
def results():
    user = get_session_user()
    poll = get_or_create_active_poll()

    # --- Muziekprofiel op basis van jouw suggesties (actieve editie) ---
    genre_counts = {}
    genre_percentages = {}

    if user:
        rows = (
            db.session.query(Genres.name, func.count())
            .join(ArtistGenres, ArtistGenres.genre_id == Genres.id)
            .join(Artists, Artists.id == ArtistGenres.artist_id)
            .join(SuggestionFeedback, SuggestionFeedback.artist_id == Artists.id)
            .filter(SuggestionFeedback.user_id == user.id)
            .filter(
                SuggestionFeedback.user_id == user.id,
                SuggestionFeedback.festival_id == poll.festival_id,
            )
            .group_by(Genres.name)
            .all()
        )
        genre_counts = {g: c for g, c in rows}
        total = sum(genre_counts.values()) or 1

        genre_percentages = {
            g: round(c * 100 / total, 1)
```

Algorithmic Support

app > routes > poll.py > results

```
139    def results():
165            g: round(c * 100 / total, 1)
166            for g, c in genre_counts.items()
167        }
168
169        # 🔥 Sorteer percentages aflopend
170        sorted_genres = sorted(
171            genre_percentages.items(),
172            key=lambda x: x[1],
173            reverse=True
174        )
175
176    # --- Jouw stemmen binnen de actieve editie ---
177    user_votes = []
178    if user and poll:
179        user_votes = (
180            db.session.query(VotesFor, Polloption, Poll, FestivalEdition, Artists)
181            .join(Polloption, VotesFor.polloption_id == Polloption.id)
182            .join(Poll, Polloption.poll_id == Poll.id)
183            .join(FestivalEdition, Poll.festival_id == FestivalEdition.id)
184            .join(Artists, Polloption.artist_id == Artists.id)
185            .filter(
186                VotesFor.user_id == user.id,
187                Poll.festival_id == poll.festival_id,
188            )
189            .order_by(FestivalEdition.Start_date.desc())
190            .all()
191        )
192
```

Algorithmic Support

app > services > genre_profile.py > generate_poll_for_user

```python
def generate_poll_for_user(user_id: int, num_options: int = 5):
    """Generate a personalized set of poll options for a user."""
    profile = get_user_genre_profile(user_id)
    proximity_scores = genre_proximity_scores(profile.keys())
    genre_id_lookup = {
        name: gid for gid, name in Genres.query.with_entities(Genres.id, Genres.name).all()
    }

    rows = (
        db.session.query(
            Artists.id,
            Artists.Artist_name,
            Genres.name.label("genre"),
        )
        .join(ArtistGenres, ArtistGenres.artist_id == Artists.id)
        .join(Genres, Genres.id == ArtistGenres.genre_id)
        .all()
    )

    artist_data = {}
    for artist_id, artist_name, genre in rows:
        if artist_id not in artist_data:
            artist_data[artist_id] = {
                "artist": artist_name,
                "genres": [],
            }
        artist_data[artist_id]["genres"].append(genre)
```

Algorithmic Support

app > services > genre_profile.py > generate_poll_for_user

```
29  def generate_poll_for_user(user_id: int, num_options: int = 5):
56
57      scored = []
58      for artist_id, data in artist_data.items():
59          genre_score = sum(profile.get(g, 0) for g in data["genres"])
60
61          # Calculate how close this artist's genres are to the user's favourites.
62          proximity = max(
63              (
64                  proximity_scores.get(genre_id_lookup.get(genre_name, -1), 0.0)
65                  for genre_name in data["genres"]
66              ),
67              default=0.0,
68          )
69
70          # Did user suggest it?
71          self_suggested = (
72              SuggestionFeedback.query
73              .filter_by(user_id=user_id, artist_id=artist_id)
74              .first()
75              is not None
76          )
77
78          score = 3 * int(self_suggested) + 2 * genre_score + 5 * proximity
79
80          scored.append((score, artist_id, data["artist"]))
81
82      scored.sort(reverse=True)
```